

**Министерство науки и высшего образования Российской
Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО"
(Университет ИТМО)**

Факультет Инфокоммуникационных технологий

**Образовательная программа Интеллектуальные системы в
гуманитарной сфере**

**Направление подготовки 45.03.04 Интеллектуальные системы в
гуманитарной сфере**

**Отчет
Курсовой работе
по предмету "Мобильная разработка (Android/iOS)"**

**Обучающиеся:
Жижилева Арина Владимировна**

**Преподаватель: Шатинский Григорий
Сергеевич**

Санкт-Петербург

2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 Разработка базовой структуры Android-приложения с навигацией (Лабораторная работа №1).....	6
1.1 Цель и задачи работы, функциональные требования.....	6
1.2. Архитектура приложения и точка входа.....	6
1.3. Архитектура приложения и точка входа.....	7
1.4 Реализация навигации (NavController и Navigation Graph).....	8
1.5 Разметка главной активности.....	8
1.6 Реализация экранов (Fragment).....	9
1.6.1 Экран "Лента".....	9
1.6.2 Экран "Профиль".....	11
1.6.3 Экран "Настройки".....	11
1.7 Жизненный цикл компонентов.....	14
1.8 Нефункциональные требования.....	15
1.9 Вывод.....	15
2 Реализация MVVM и управление состоянием приложения.....	16
2.1 Цель и задачи главы.....	16
2.2 Выбор архитектуры и обоснование.....	16
2.3 Реализация ViewModel.....	17
2.3.1 ProfileViewModel.....	17
2.4 Реализация UI и связывание с ViewModel.....	18
2.4.1 Экран "Профиль".....	18
2.4.2 Экран "Настройки".....	20
2.5 Сохранение состояния при повороте экрана.....	21
2.6 Логирование жизненного цикла ViewModel.....	21
2.7 Выводы по главе 2.....	21
3 Разработка клиентского Android-приложения мессенджера.....	23
3.1 Цель и задачи лабораторной работы.....	23
3.2 Архитектура приложения.....	23
3.3 Реализация сетевого взаимодействия.....	24
3.4 Модель данных.....	25
3.5 Локальное хранилище данных.....	26
3.6 Репозиторий данных.....	28
3.7 ViewModel и управление состоянием.....	29
4 Улучшение пользовательского интерфейса и фоновая синхронизация	

данных.....	35
4.1 Кастомизация списка сообщений и использование Material Design.....	35
4.2 Реализация возможности лайка сообщений.....	37
4.3 Использование компонентов Material Design.....	39
4.4 Фоновая синхронизация данных с использованием WorkManager...	40
4.5 Реализация уведомлений.....	41
4.6 Запрос разрешений и обработка системных событий.....	43
4.7 Выводы.....	44
ЗАКЛЮЧЕНИЕ.....	45

ВВЕДЕНИЕ

Мобильная разработка занимает одну из ведущих позиций в современной IT-индустрии, поскольку смартфоны стали основным средством потребления контента и общения. Создание качественного мессенджера требует от разработчика не только навыков в дизайне интерфейса, но и глубокого понимания архитектуры, асинхронного программирования и управления данными.

Актуальность данной работы определяется необходимостью изучения и практического применения современных подходов к разработке под Android - самой популярной мобильной платформы в мире. Основная цель курсового проекта заключается в проектировании и реализации прототипа мобильного мессенджера с использованием актуального технологического стека. В процессе выполнения проекта решается комплекс задач, связанных с построением гибкой и масштабируемой архитектуры приложения. Важным аспектом является переход от простых монолитных решений к использованию паттерна MVVM, который позволяет распределить ответственность между компонентами и повысить тестируемость кода.

Особое внимание уделяется пользовательскому опыту: в современных условиях он невозможен без быстрой навигации, понятных элементов управления и поддержки различных режимов отображения, включая темную тему.

Ключевой частью современной мобильной разработки является обеспечение надежности хранения и передачи данных. Введение подчеркивает, что приложение не может полностью зависеть от стабильного интернет-соединения. Пользователи ожидают, что приложение будет работать даже при отсутствии сети. Поэтому в проекте рассматривается концепция Offline-first, предполагающая наличие устойчивого локального кэша.

Использование реляционных баз данных на мобильном устройстве и их синхронизация с внешними серверами через REST API являются важными навыками для разработчика. Эти технологии позволяют создавать продукты, которые эффективно используют ресурсы устройства и обеспечивают сохранность данных.

Кроме того, в работе рассматриваются механизмы фоновое выполнения задач и уведомлений, которые являются связующим звеном между приложением и пользователем. В условиях высокой конкуренции на рынке мобильного ПО важно удерживать внимание аудитории и предоставлять актуальную информацию в режиме реального времени. Изучение инструментов фоновой синхронизации и системы уведомлений позволяет автоматизировать обновление контента. В совокупности все технологии и подходы, рассматриваемые в курсовом проекте, формируют целостное представление о процессе разработки мобильного продукта: от проектирования структуры экрана до создания системы, способной работать в фоне и эффективно взаимодействовать с облачными сервисами.

1 Разработка базовой структуры Android-приложения с навигацией (Лабораторная работа №1)

1.1 Цель и задачи работы, функциональные требования

Целью первой лабораторной работы являлось создание простого прототипа Android-приложения "Мессенджер" с использованием базовых компонентов платформы: Activity, Fragment и встроенной навигации. Данный этап позволил закрепить навыки построения архитектуры приложения на основе нескольких экранов, а также понять принципы взаимодействия компонентов и жизненного цикла Activity/Fragment. В рамках задания была реализована основная структура приложения, обеспечивающая дальнейшее расширение функционала в следующих лабораторных работах.

В соответствии с заданием, приложение должно содержать одну главную активность, которая является точкой входа, и реализовывать навигацию между экранами через Bottom Navigation. Каждая вкладка панели открывает отдельный экран:

1. экран новостной ленты (пока содержит заглушки сообщений),
2. экран профиля пользователя с отображением данных,
3. экран настроек с переключателем темы.

Навигация реализована с использованием стандартных инструментов Android Navigation: NavController и Navigation Graph. Кроме того, требовалось отслеживать жизненный цикл компонентов, выводя сообщения в лог при создании и завершении работы Activity и Fragment.

1.2. Архитектура приложения и точка входа

В рамках лабораторной работы была выбрана стандартная архитектура Android-приложения, основанная на одной главной Activity и нескольких Fragment. Главная активность выступает как контейнер для фрагментов, а навигация между экранами осуществляется через Bottom Navigation и NavController.

Используемый стек технологий:

1. Kotlin - основной язык разработки;
2. Android SDK (Activity/Fragment);
3. Navigation Component (NavController, Navigation Graph);
4. Bottom Navigation для переключения экранов;
5. Logcat для отслеживания жизненного цикла компонентов.

1.3. Архитектура приложения и точка входа

Точкой входа приложения является MainActivity. В ней происходит инициализация навигации и привязка BottomNavigationView к NavController, что обеспечивает переключение между фрагментами при выборе вкладок.

```
class MainActivity : AppCompatActivity() {
    3 Usages
    private lateinit var binding: ActivityMainBinding
    6 Usages
    private val TAG = "MainActivity"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, msg = "onCreate")
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        val navHostFragment = supportFragmentManager
            .findFragmentById(id = R.id.nav_host_fragment) as NavHostFragment

        val navController = navHostFragment.navController
        binding.bottomNav.setupWithNavController(navController)
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, msg = "onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, msg = "onResume")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, msg = "onPause")
    }
}
```

Рисунок 1 - MainActivity (настройка NavController и BottomNavigation).

1.4 Реализация навигации (NavController и Navigation Graph)

Для управления навигацией в приложении был использован Navigation Component. Все фрагменты и связи между ними описаны в nav_graph.xml, что позволяет централизованно управлять маршрутами и переходами. Такой подход повышает читаемость кода и упрощает дальнейшее расширение приложения.

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/feedFragment">

    <fragment
        android:id="@+id/feedFragment"
        android:name="com.example.messengerlab.ui.feed.FeedFragment"
        android:label="Лента" />

    <fragment
        android:id="@+id/profileFragment"
        android:name="com.example.messengerlab.ui.profile.ProfileFragment"
        android:label="Профиль" />

    <fragment
        android:id="@+id/settingsFragment"
        android:name="com.example.messengerlab.ui.settings.SettingsFragment"
        android:label="Настройки" />
</navigation>
```

Рисунок 2 - Navigation Graph (NavController + фрагменты).

1.5 Разметка главной активности

В activity_main.xml была добавлена разметка с контейнером для фрагментов (NavHostFragment) и нижней панелью BottomNavigationView. Такая структура позволяет разделить интерфейс на независимые экраны, каждый из которых реализован в виде отдельного Fragment.


```

<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment"
        android:name="androidx.navigation.fragment.NavHostFragment"
        app:navGraph="@navigation/nav_graph"
        app:defaultNavHost="true"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_nav"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom"
        app:menu="@menu/bottom_nav_menu" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

Рисунок 3 - activity_main.xml (BottomNavigationView).

1.6 Реализация экранов (Fragment)

1.6.1 Экран "Лента"

Экран "Лента" реализован как FeedFragment. В рамках лабораторной работы он содержит заглушки сообщений (3 статических элемента), что позволяет проверить корректность навигации и отображения контента.

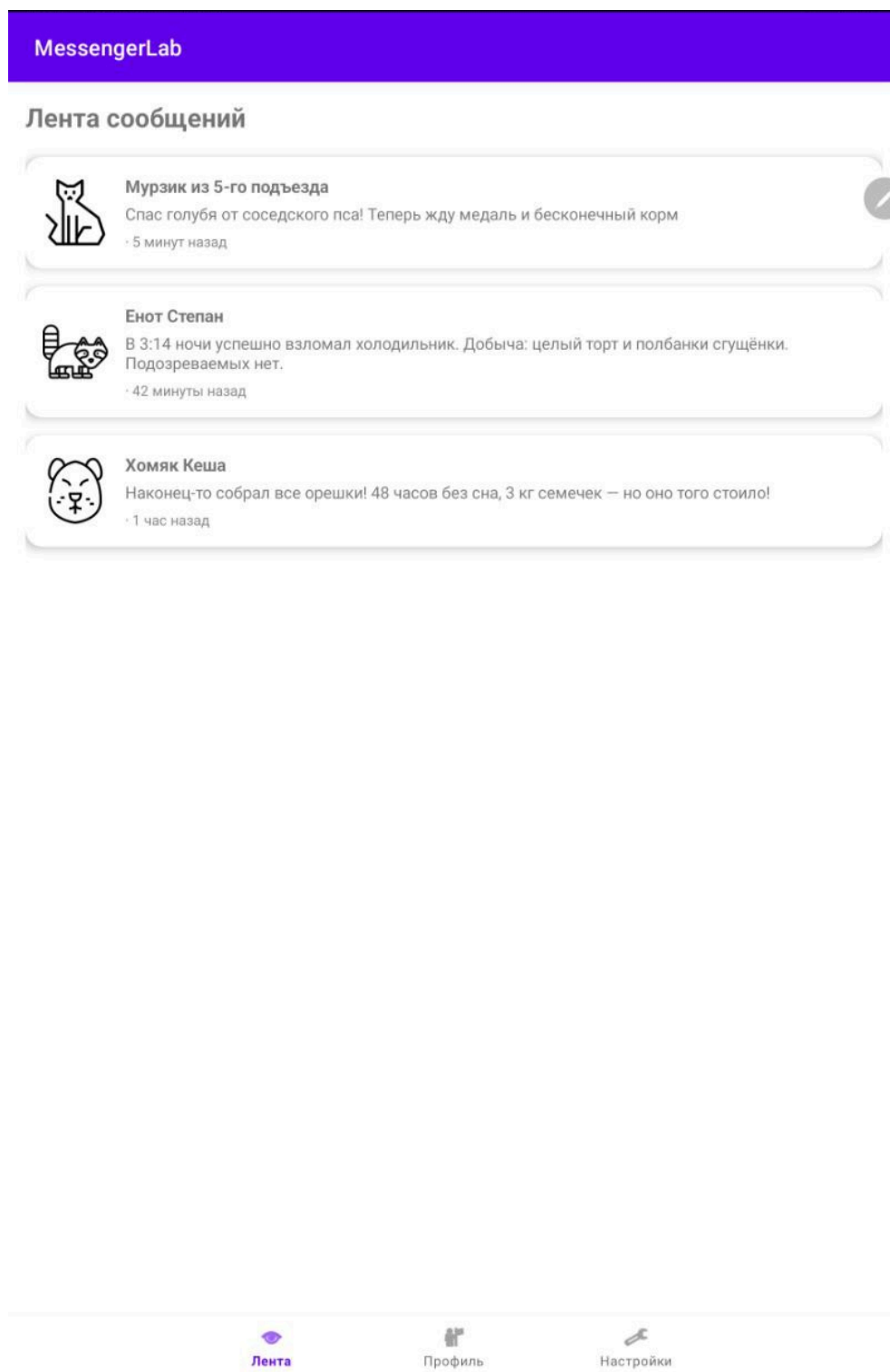


Рисунок 4 - Экран "Лента" (3 заглушки сообщений).

1.6.2 Экран "Профиль"

Экран "Профиль" реализован как ProfileFragment. На данном экране отображаются основные данные пользователя в текстовом виде, что соответствует требованиям задания.



Рисунок 5 - Экран "Профиль" (отображение данных пользователя).

1.6.3 Экран "Настройки"

Экран "Настройки" реализован как SettingsFragment. В рамках лабораторной работы на экране присутствует базовый элемент интерфейса - переключатель темы (Dark/Light), который позволяет изменить внешний вид приложения.



Рисунок 6 - Экран "Настройки" (Light для темы).

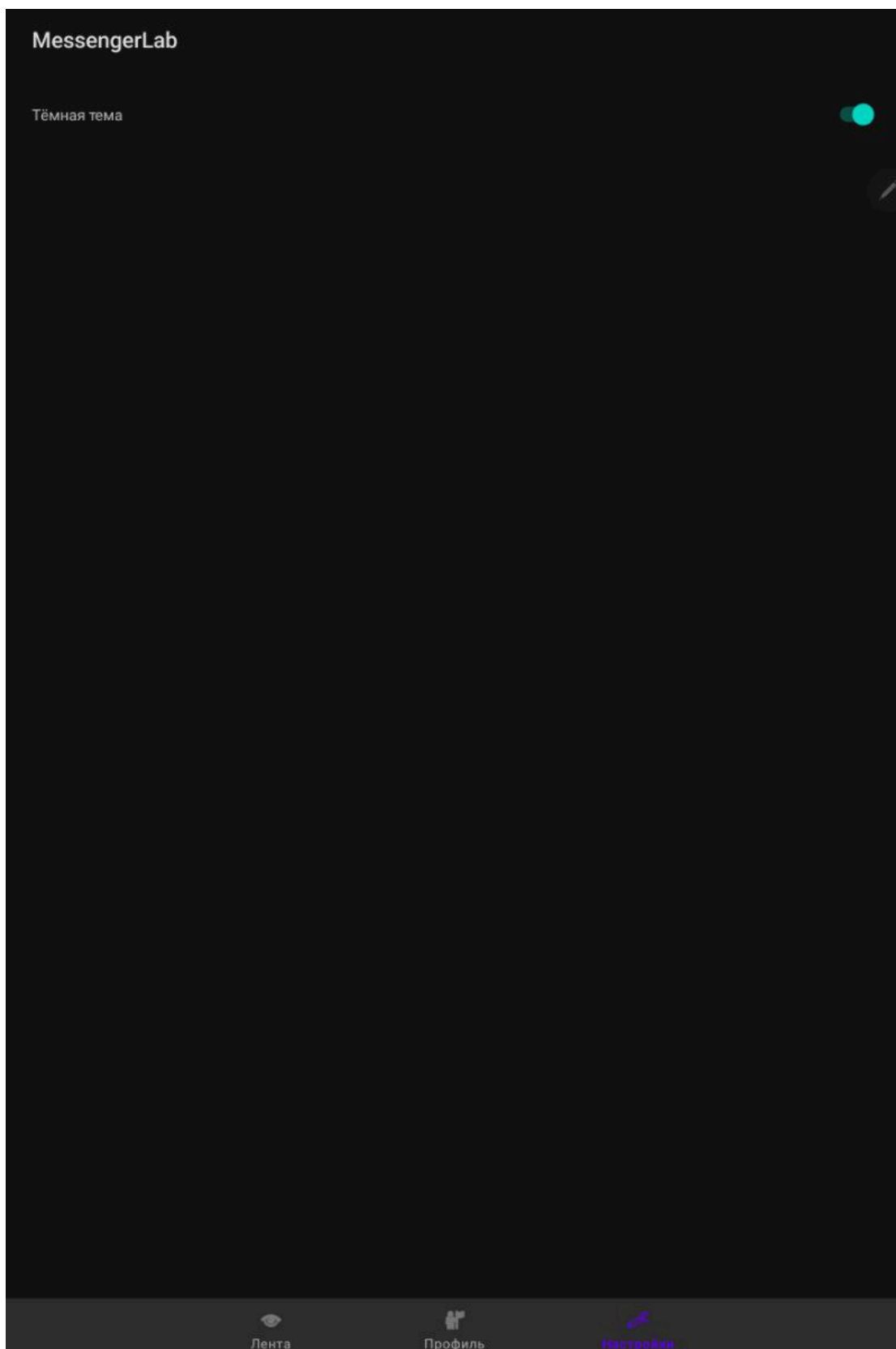


Рисунок 7 - Экран "Настройки" (Dark для темы).

1.7 Жизненный цикл компонентов

В целях выполнения требований лабораторной работы была реализована логика отслеживания жизненного цикла Activity и Fragment. В методах onCreate, onStart, onResume, onPause, onStop и onDestroy выводятся сообщения в лог. Это позволяет отслеживать поведение приложения при запуске, сворачивании и закрытии, а также обеспечивает понимание порядка вызовов методов жизненного цикла.

```
override fun onStart() {  
    super.onStart()  
    Log.d(TAG, msg = "onStart")  
}  
  
override fun onResume() {  
    super.onResume()  
    Log.d(TAG, msg = "onResume")  
}  
  
override fun onPause() {  
    super.onPause()  
    Log.d(TAG, msg = "onPause")  
}  
  
override fun onStop() {  
    super.onStop()  
    Log.d(TAG, msg = "onStop")  
}  
  
override fun onDestroy() {  
    super.onDestroy()  
    Log.d(TAG, msg = "onDestroy")  
}
```

Рисунок 8 - Логи жизненного цикла (MainActivity).

1.8 Нефункциональные требования

Код приложения был структурирован с учетом дальнейшего расширения функционала. Использование Navigation Component и разделение интерфейса на отдельные фрагменты позволяет добавлять новые экраны и функции без значительного изменения существующей архитектуры. Приложение запускается без ошибок, интерфейс интуитивно понятен, а навигация между вкладками осуществляется плавно и корректно.

1.9 Вывод

В результате выполнения первой лабораторной работы был создан базовый каркас приложения "Мессенджер", включающий одну главную Activity и три экрана, реализованные как Fragment. Были освоены основные инструменты Android Navigation и жизненный цикл компонентов. Полученная структура является надежной основой для дальнейшей разработки и расширения функционала в следующих лабораторных работах.

2 Реализация MVVM и управление состоянием приложения

2.1 Цель и задачи главы

Целью данной главы является разработка механизма управления состоянием приложения с использованием архитектурного паттерна MVVM (Model–View–ViewModel) и обеспечение сохранения пользовательских данных при изменении конфигурации устройства (например, поворот экрана). В рамках решения поставленной цели необходимо выполнить следующие задачи:

1. Реализовать ViewModel для хранения состояния экрана "Профиль" и экрана "Настройки".
2. Обеспечить возможность редактирования данных пользователя (имя и статус) и их сохранение во ViewModel.
3. Реализовать переключение темы приложения (светлая/темная) с сохранением состояния.

Гарантировать сохранение данных при повороте экрана.

4. Логировать этапы жизненного цикла ViewModel.

2.2 Выбор архитектуры и обоснование

Для реализации управления состоянием приложения был выбран паттерн MVVM, который позволяет разделить ответственность между UI и логикой приложения. В данном подходе:

1. View (Fragment) отвечает только за отображение данных и обработку пользовательских действий.
2. ViewModel хранит состояние интерфейса и выполняет обработку логики, не зависящей от жизненного цикла View.
3. LiveData обеспечивает реактивное обновление UI при изменении данных.

Такой подход повышает тестируемость, упрощает поддержку кода и обеспечивает устойчивость приложения к изменениям конфигурации устройства.

2.3 Реализация ViewModel

2.3.1 ProfileViewModel

Для экрана "Профиль" был создан класс ProfileViewModel, который содержит два поля name и status типа LiveData<String>. Данные обновляются через методы updateName() и updateStatus(). Для фиксации жизненного цикла во ViewModel реализованы методы init и onCleared(), выводящие сообщения в лог.

```
class ProfileViewModel : ViewModel() {  
  
    2 Usages  
    private val _name = MutableLiveData<String>() { value = "" }  
    val name: LiveData<String> get() = _name  
  
    2 Usages  
    private val _status = MutableLiveData<String>() { value = "" }  
    2 Usages  
    val status: LiveData<String> get() = _status  
  
    1 Usage  
    fun updateName(new: String) {  
        | _name.value = new  
    }  
  
    1 Usage  
    fun updateStatus(new: String) {  
        | _status.value = new  
    }  
  
    init {  
        | Log.d( tag = "ProfileVM", msg = "Создан ProfileViewModel (один раз з  
    }  
  
    override fun onCleared() {  
        | Log.d( tag = "ProfileVM", msg = "ProfileViewModel уничтожен (когда з  
        | super.onCleared()  
    }  
}
```

Рисунок 9 - ProfileViewModel.

2.3.2 SettingsViewModel

Для экрана "Настройки" реализован SettingsViewModel, который хранит состояние переключателя темы (isDarkTheme). При изменении значения вызывается AppCompatActivity.setDefaultNightMode(), что обеспечивает применение темы в приложении. Также фиксируются этапы жизненного цикла через Log.

```
class SettingsViewModel : ViewModel() {  
  
    2 Usages  
    private val _isDarkTheme = MutableLiveData(value = false)  
    1 Usage  
    val isDarkTheme: LiveData<Boolean> get() = _isDarkTheme  
  
    1 Usage  
    fun setTheme(isDark: Boolean) {  
        _isDarkTheme.value = isDark  
        AppCompatActivity.setDefaultNightMode(  
            if (isDark) AppCompatActivity.MODE_NIGHT_YES  
            else AppCompatActivity.MODE_NIGHT_NO  
        )  
    }  
  
    init {  
        Log.d(tag = "SettingsVM", msg = "Создан SettingsViewModel (один раз)  
    }  
  
    override fun onCleared() {  
        Log.d(tag = "SettingsVM", msg = "SettingsViewModel уничтожен (когда  
        super.onCleared()  
    }  
}
```

Рисунок 10 - SettingsViewModel.

2.4 Реализация UI и связывание с ViewModel

2.4.1 Экран "Профиль"

В ProfileFragment осуществляется подписка на LiveData из ProfileViewModel. При изменении значения поля name или status

обновляется UI, а при вводе текста данные отправляются обратно во ViewModel.

Фрагмент "Профиль" реализован следующим образом:

```
override fun onCreateView(view: View, savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)
    Log.d(TAG, msg = "onViewCreated")

    vm.name.observe(viewLifecycleOwner) { name ->
        if (binding.etName.text.toString() != name) {
            binding.etName.setText(name)
        }
        binding.tvResult.text = "Имя: $name\nСтатус: ${vm.status.value ?: ""}"
    }

    vm.status.observe(viewLifecycleOwner) { status ->
        if (binding.etStatus.text.toString() != status) {
            binding.etStatus.setText(status)
        }
        binding.tvResult.text = "Имя: ${vm.name.value ?: ""}\nСтатус: $status"
    }

    binding.etName.doOnTextChanged { text, _, _, _ ->
        vm.updateName(new = text.toString())
    }

    binding.etStatus.doOnTextChanged { text, _, _, _ ->
        vm.updateStatus(new = text.toString())
    }
}
```

Рисунок 11 - ProfileFragment.



Рисунок 12 - Профиль.

2.4.2 Экран "Настройки"

В `SettingsFragment` осуществляется подписка на `isDarkTheme`. При изменении переключателя вызывается метод `setTheme()` `ViewModel`.

```

override fun onCreateView(view: View, savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)

    viewModel.isDarkTheme.observe(viewLifecycleOwner) { isDark ->
        binding.switchTheme.isChecked = isDark
    }

    binding.switchTheme.setOnCheckedChangeListener { _, isChecked ->
        viewModel.setTheme(isChecked)
    }
}

```

Рисунок 13 - SettingsFragment.

2.5 Сохранение состояния при повороте экрана

Поскольку ViewModel привязан к жизненному циклу Activity/Fragment и сохраняется при пересоздании UI, введенные данные не теряются при повороте экрана. После изменения ориентации экрана Fragment пересоздается, однако ViewModel остается прежним, что обеспечивает сохранение состояния.

2.6 Логирование жизненного цикла ViewModel

Для контроля создания и уничтожения ViewModel в классах ProfileViewModel и SettingsViewModel реализованы методы init и onCleared(), которые выводят сообщения в лог.

2.7 Выводы по главе 2

В ходе выполнения второй лабораторной работы была реализована архитектура MVVM, обеспечивающая разделение ответственности между UI и логикой приложения. В результате:

1. состояние профиля и настроек хранится во ViewModel;
2. UI обновляется реактивно через LiveData;
3. данные сохраняются при повороте экрана;
4. жизненный цикл ViewModel контролируется через логирование.

Данная реализация повышает масштабируемость проекта и обеспечивает основу для дальнейшего развития функционала.

3 Разработка клиентского Android-приложения мессенджера

3.1 Цель и задачи лабораторной работы

Целью третьей лабораторной работы является разработка клиентского Android-приложения мессенджера, реализующего загрузку сообщений с удаленного сервера, их локальное хранение и отображение пользователю с возможностью обновления данных.

Для достижения поставленной цели были сформулированы следующие задачи:

1. реализовать сетевое взаимодействие с REST API;
2. организовать локальное хранение данных с использованием базы данных Room;
3. применить архитектурный паттерн MVVM;
4. обеспечить обновление данных по запросу пользователя;
5. реализовать пользовательский интерфейс для отображения списка сообщений.

3.2 Архитектура приложения

В разработанном приложении используется архитектурный паттерн MVVM (Model–View–ViewModel), обеспечивающий разделение ответственности между слоями приложения и упрощающий сопровождение кода.

Архитектура включает следующие уровни:

1. Data layer - отвечает за работу с сетью и локальной базой данных;
2. Domain / Repository layer - объединяет источники данных и инкапсулирует бизнес-логику;
3. UI layer - реализует пользовательский интерфейс и отображение данных.

Общая структура проекта организована по модульному принципу, что повышает читаемость и масштабируемость приложения.

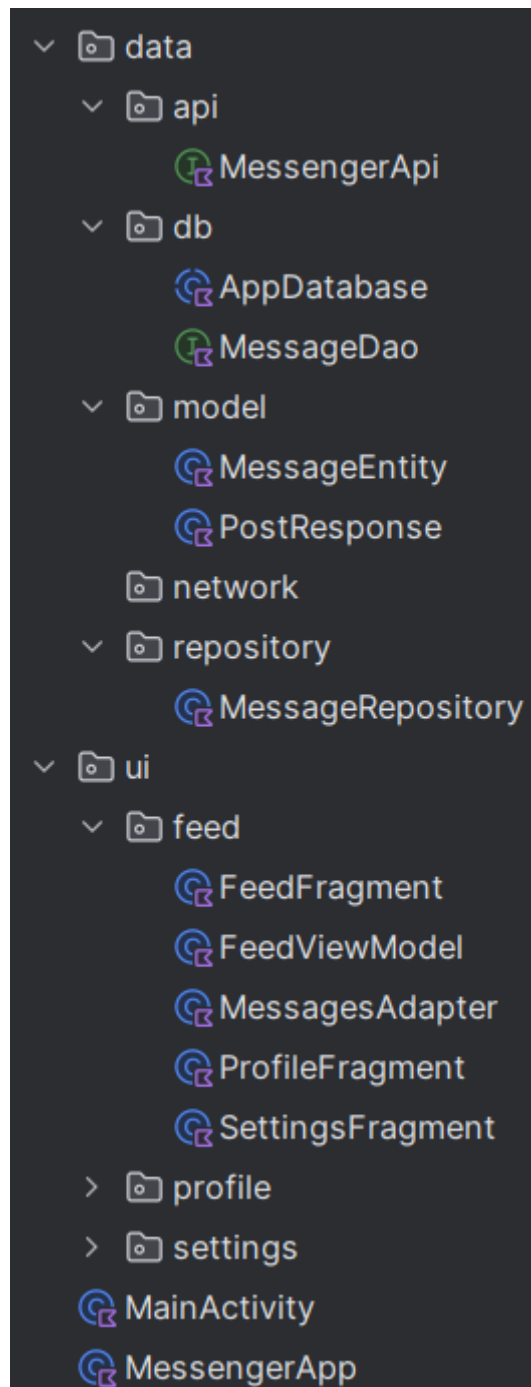


Рисунок 14 - Структура проекта Android-приложения.

3.3 Реализация сетевого взаимодействия

Для получения сообщений с сервера используется библиотека Retrofit. Взаимодействие с API описано в интерфейсе MessengerApi.

Сетевой запрос реализован с помощью HTTP-метода GET и возвращает объект типа PostResponse, содержащий список сообщений.

Ответ от сервера автоматически преобразуется из формата JSON в объекты Kotlin с использованием библиотеки Gson.

```
package com.example.messengerlab.data.api

import ...

interface MessengerApi {

    @GET(value = "posts")
    suspend fun getMessages(): Response<PostResponse>
}
```

Рисунок 15 - Интерфейс для работы с REST API.

3.4 Модель данных

Для хранения сообщений используется единая модель MessageEntity, которая одновременно применяется:

1. для десериализации сетевых данных;
2. для хранения информации в локальной базе данных Room.

Модель содержит следующие поля:

1. идентификатор сообщения;
2. имя автора;
3. текст сообщения;
4. временную метку.

Аннотация @Entity используется для определения таблицы базы данных, а @PrimaryKey - для задания первичного ключа.

```

package com.example.messengerlab.data.model

import ...

@Entity(tableName = "messages")
data class MessageEntity(
    @PrimaryKey
    val id: Int,

    @SerializedName(value = "title")
    val author: String,

    @SerializedName(value = "body")
    val text: String,

    val timestamp: String = "12:00"

```

Рисунок 16 - Модель данных сообщения.

3.5 Локальное хранилище данных

Для реализации офлайн-доступа используется библиотека Room.

Доступ к данным осуществляется через интерфейс MessageDao, содержащий методы:

1. получения всех сообщений;
2. вставки списка сообщений;
3. очистки таблицы сообщений.

База данных описана в абстрактном классе AppDatabase, который наследуется от RoomDatabase и содержит ссылку на DAO.

Использование Room позволяет обеспечить надежное хранение данных и работу приложения при отсутствии сетевого соединения.

```

package com.example.messengerlab.data.db

import ...

7 Usages 1 Implementation
@Dao
interface MessageDao {
    1 Usage 1 Implementation
    @Query(value = "SELECT * FROM messages")
    suspend fun getAllMessages(): List<MessageEntity>

    1 Usage 1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(messages: List<MessageEntity>)

    1 Usage 1 Implementation
    @Query(value = "DELETE FROM messages")
    suspend fun clearAll()

```

Рисунок 17 - DAO-интерфейс для работы с базой данных.

```

package com.example.messengerlab.data.db

import ...

3 Usages 1 Implementation
@Database(entities = [MessageEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    1 Usage 1 Implementation
    abstract fun messageDao(): MessageDao
}

```

Рисунок 18 - Конфигурация базы данных Room.

3.6 Репозиторий данных

Класс `MessageRepository` реализует паттерн `Repository` и является единой точкой доступа к данным.

Репозиторий выполняет следующие функции:

1. загрузку сообщений с сервера;
2. сохранение полученных данных в локальную базу;
3. предоставление данных пользовательскому интерфейсу.

При успешном сетевом запросе база данных очищается и заполняется актуальными данными. В случае ошибки загрузки приложение продолжает использовать данные из локального хранилища.

```
class MessageRepository(  
    private val api: MessengerApi,  
    private val dao: MessageDao  
) {  
    1 Usage  
    suspend fun refreshMessages() {  
        try {  
            val response = api.getMessages()  
            if (response.isSuccessful) {  
                val messages = response.body()?.posts ?: emptyList()  
                Log.d(tag = "REPO_DEBUG", msg = "Успешно загружено: ${message}")  
  
                dao.clearAll()  
                dao.insertAll(messages)  
            }  
        } catch (e: Exception) {  
            Log.e(tag = "REPO_DEBUG", msg = "Ошибка загрузки: ${e.message}")  
        }  
    }  
}  
  
1 Usage  
suspend fun getAllMessages(): List<MessageEntity> = dao.getAllMessages()  
}
```

Рисунок 19 - Репозиторий для управления источниками данных.

3.7 ViewModel и управление состоянием

Для управления состоянием пользовательского интерфейса используется класс FeedViewModel.

ViewModel содержит:

1. LiveData со списком сообщений;
2. состояние загрузки данных;
3. обработку ошибок.

Загрузка данных осуществляется в корутине с использованием viewModelScope, что позволяет выполнять асинхронные операции без блокировки основного потока.

```
class FeedViewModel(private val repository: MessageRepository) : ViewModel() {

    2 Usages
    private val _messages = MutableLiveData<List<MessageEntity>>()
    1 Usage
    val messages: LiveData<List<MessageEntity>> = _messages

    1 Usage
    private val _isLoading = MutableLiveData<Boolean>()
    1 Usage
    val isLoading: LiveData<Boolean> = _isLoading

    1 Usage
    private val _error = MutableLiveData<String?>()
    1 Usage
    val error: LiveData<String?> = _error

    init {
        loadMessages(forceReload = false)
    }

    2 Usages
    fun loadMessages(forceReload: Boolean = false) {
        viewModelScope.launch {
            try {
                repository.refreshMessages()
                val list = repository.getAllMessages()
                _messages.value = list
            } catch (e: Exception) {
            }
        }
    }
}
```

Рисунок 20 - ViewModel экрана ленты сообщений.

3.8 Пользовательский интерфейс

Отображение списка сообщений реализовано во фрагменте FeedFragment.

Для отображения списка используется компонент RecyclerView с адаптером MessagesAdapter.

Каждое сообщение отображается в отдельной карточке, содержащей имя автора и текст сообщения.

Пользовательский интерфейс также включает:

1. кнопку обновления данных;
2. индикатор загрузки.

Интерфейс реализован с использованием ConstraintLayout и Material Components.

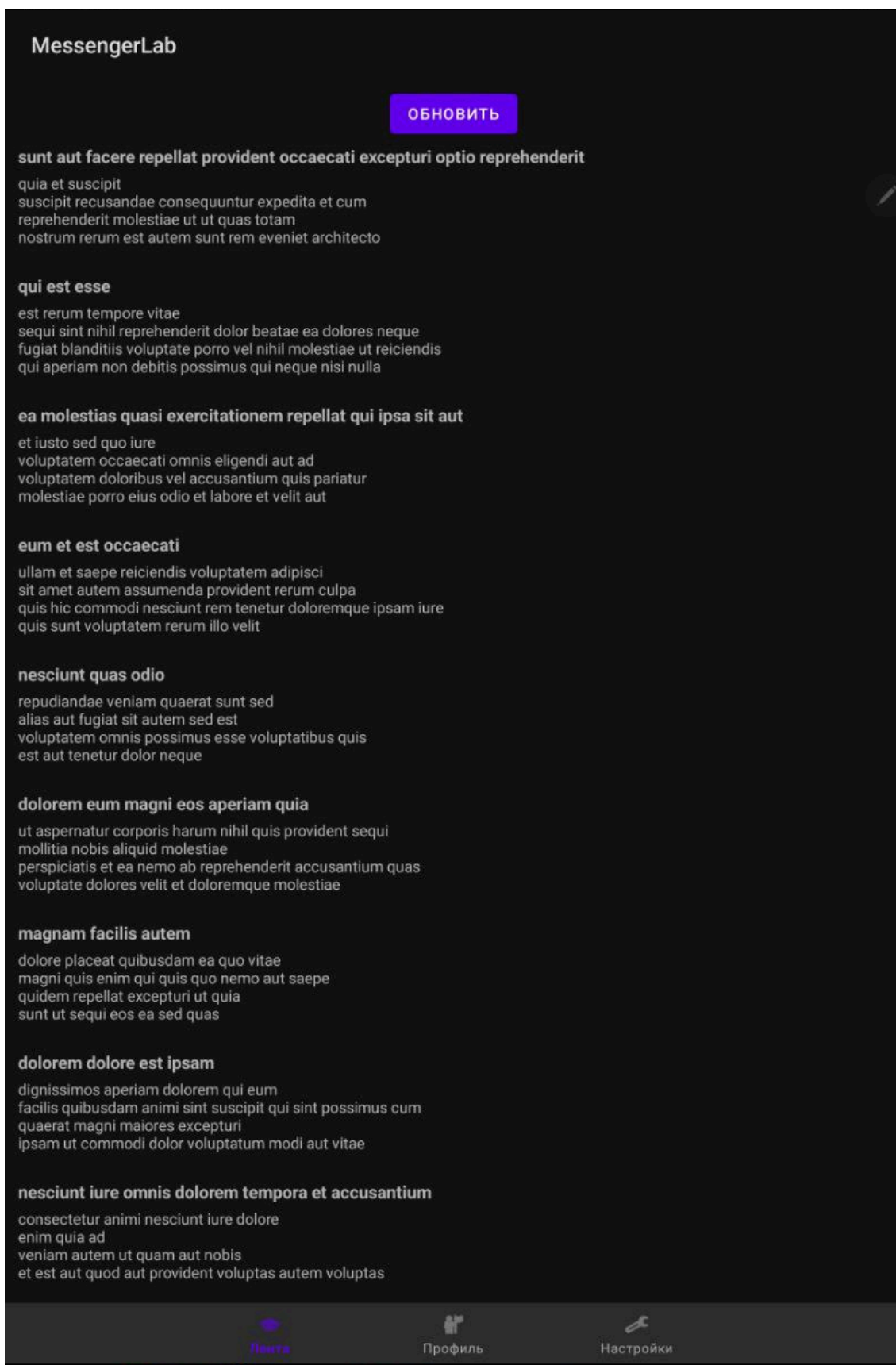


Рисунок 21 - Экран приложения.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:id="@+id/tvAuthor"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Author Name"
            android:textStyle="bold"
            android:textSize="16sp"
            android:layout_marginBottom="4dp"/>

        <TextView
            android:id="@+id/tvMessage"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Message body goes here..."
            android:textSize="14sp"/>

    </LinearLayout>

</androidx.cardview.widget.CardView>

```

Рисунок 22 - Макет элемента списка сообщений.

3.9 Инициализация приложения

Класс `MessengerApp`, наследующийся от `Application`, используется для инициализации:

1. базы данных Room;
2. Retrofit API;
3. репозитория данных.

Данный подход позволяет централизованно управлять зависимостями и использовать их в различных частях приложения.

```
class MessengerApp : Application() {  
  
    1 Usage  
    private val database by lazy {  
        Room.databaseBuilder(applicationContext, class = AppDatabase::class.java)  
    }  
  
    // API  
    1 Usage  
    private val api by lazy {  
        Retrofit.Builder()  
            .baseUrl(baseUrl = "https://jsonplaceholder.typicode.com/")  
            .addConverterFactory(factory = GsonConverterFactory.create())  
            .build()  
            .create(MessengerApi::class.java)  
    }  
  
    // Репозиторий  
    1 Usage  
    val repository by lazy {  
        MessageRepository(api, dao = database.messageDao())  
    }  
}
```

Рисунок 23 - Инициализация базы данных и сетевого клиента.

3.10 Выводы по главе

В ходе выполнения третьей лабораторной работы было разработано Android-приложение мессенджера с использованием современных инструментов и архитектурных подходов.

В результате работы:

1. реализовано сетевое взаимодействие с REST API;
2. обеспечено локальное хранение данных;
3. применена архитектура MVVM;
4. реализован удобный и отзывчивый пользовательский интерфейс.

Разработанное приложение соответствует поставленным требованиям и может быть расширено дополнительным функционалом, таким как отправка сообщений, авторизация пользователей и push-уведомления.

4 Улучшение пользовательского интерфейса и фоновая синхронизация данных

4.1 Кастомизация списка сообщений и использование Material Design

В рамках данной лабораторной работы был переработан экран новостной ленты.

Для отображения сообщений реализован кастомный RecyclerView.Adapter, использующий компонент CardView и элементы Material Design.

Каждое сообщение отображает:

1. аватар пользователя;
2. имя автора;
3. заголовок сообщения;
4. текст сообщения;
5. кнопку "лайк", изменяющую состояние при нажатии.

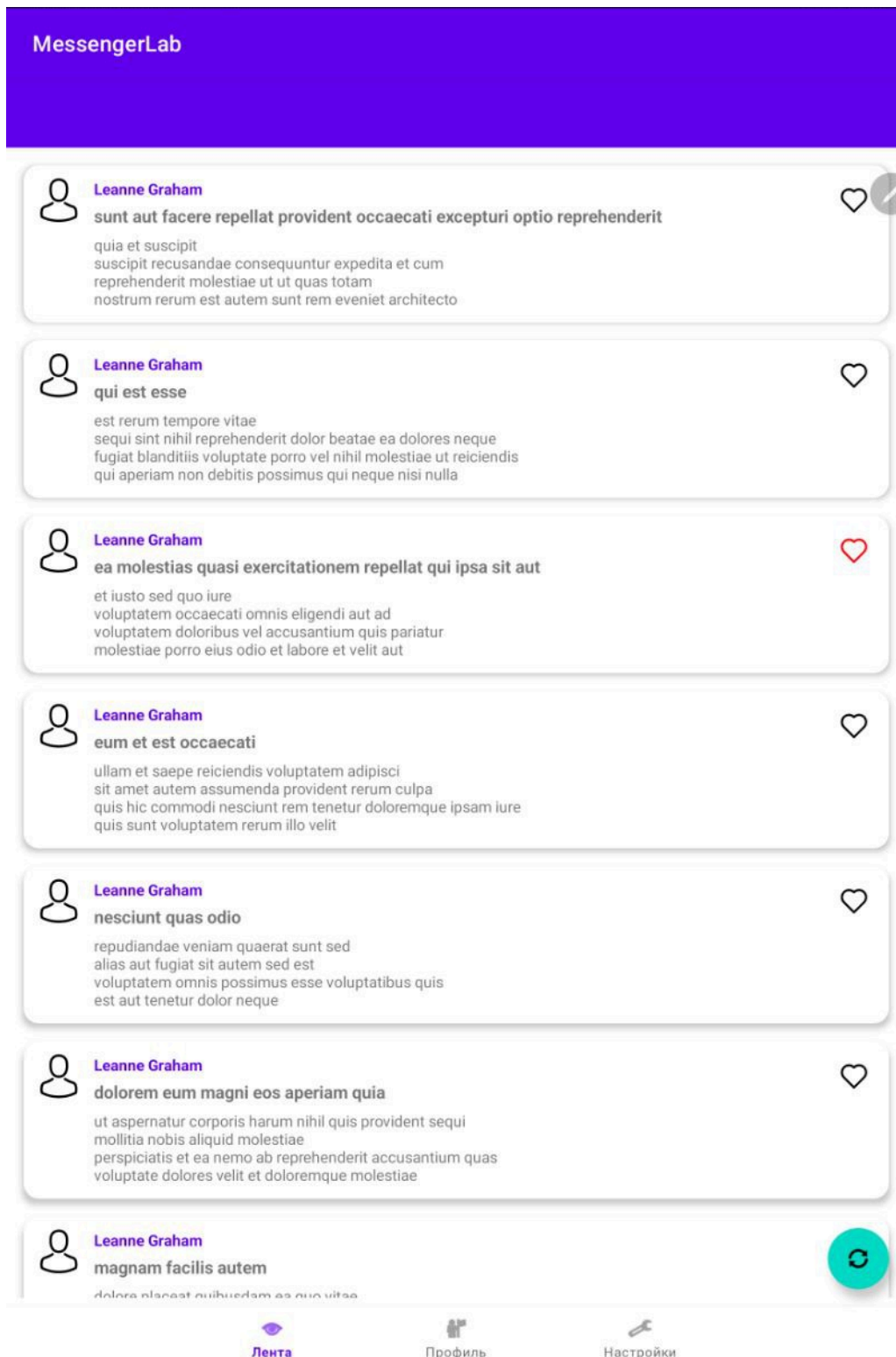


Рисунок 24 - Интерфейс.

Для реализации списка был создан кастомный адаптер `MessagesAdapter`, наследующийся от `ListAdapter` с использованием `DiffUtil`, что обеспечивает эффективное обновление элементов списка.

4 Usages

```
class MessageViewHolder(  
    private val binding: ItemMessageBinding,  
    private val onLikeClick: (MessageEntity) -> Unit  
) : RecyclerView.ViewHolder( itemView = binding.root) {  
  
    1 Usage  
  
    fun bind(message: MessageEntity) {  
        binding.tvTitle.text = message.authorName  
        binding.tvBody.text = message.body  
  
        binding.tvAuthor.text = message.authorName  
        binding.tvTitle.text = message.title  
        binding.tvBody.text = message.body  
  
        binding.btnLike.setImageResource(  
            if (message.isLiked)  
                R.drawable.ic_heart_filled  
            else  
                R.drawable.ic_heart_outline  
        )  
  
        binding.btnLike.setOnClickListener {  
            onLikeClick(message)  
        }  
    }  
}
```

Рисунок 25 - MessagesAdapter.kt.

4.2 Реализация возможности лайка сообщений

Для поддержки лайков в модель данных было добавлено булево поле `isLiked`.

Состояние лайка хранится в локальной базе данных Room и обновляется при взаимодействии пользователя с интерфейсом.

Изменение состояния выполняется через репозиторий, что соответствует архитектуре MVVM и принципу разделения ответственности.

```
@Entity(tableName = "messages")
data class MessageEntity(
    @PrimaryKey val id: Int,
    val userId: Int,
    val authorName: String,
    val title: String,
    val body: String,
    val isLiked: Boolean = false
)
```

Рисунок 26 - MessageEntity.kt.

```

@Dao
interface MessageDao {

    1 Usage 1 Implementation
    @Query(value = "SELECT * FROM messages")
    suspend fun getAllMessages(): List<MessageEntity>

    1 Usage 1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(messages: List<MessageEntity>)

    1 Usage 1 Implementation
    @Query(value = "DELETE FROM messages")
    suspend fun clearAll()

    1 Implementation
    @Query(value = "UPDATE messages SET isLiked = :isLiked WHERE id = :id")
    suspend fun updateLike(id: Int, isLiked: Boolean)

    1 Usage 1 Implementation
    @Update
    suspend fun update(message: MessageEntity)
}

```

Рисунок 27 -MessageDao.kt.

4.3 Использование компонентов Material Design

Для улучшения пользовательского интерфейса были применены компоненты Material Design:

1. CardView - для оформления сообщений;
2. AppBarLayout и MaterialToolbar - для верхней панели;
3. FloatingActionButton - для ручного обновления данных.

Использование данных компонентов повышает визуальную согласованность приложения и улучшает пользовательский опыт.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            android:title="Лента"
            android:titleTextColor="@android:color/white"
            android:background="?attr/colorPrimary"/>
        </com.google.android.material.appbar.AppBarLayout>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="8dp"
        app:layout_behavior="com.google.android.material.appbar.AppBarLayout"

    <ProgressBar
        android:id="@+id/progressBar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:visibility="gone"
        app:layout_anchor="@id/recyclerView"
        app:layout_anchorGravity="center"/>

```

Рисунок 28 - fragment_feed.xml.

4.4 Фоновая синхронизация данных с использованием WorkManager

Для автоматического обновления сообщений был реализован фоновый процесс с использованием WorkManager.

Периодическая задача выполняется при наличии интернет-соединения и обновляет локальную базу данных.

Фоновая логика реализована в классе SyncMessagesWorker, который использует CoroutineWorker.

```
# Usage
class SyncMessagesWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker( appContext = context, params) {

    1 Usage
    private val repository = (context.applicationContext as MessengerApp).rep

    override suspend fun doWork(): Result {
        if (!isOnline(applicationContext)) {
            return Result.retry()
        }

        return try {
            repository.refreshMessages()
            NotificationHelper.showSyncSuccess(applicationContext)
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }
}

1 Usage
private fun isOnline(context: Context): Boolean {
    val cm = context.getSystemService( p0 = Context.CONNECTIVITY_SERVICE)
    val network = cm.activeNetwork ?: return false
    val capabilities = cm.getNetworkCapabilities(network) ?: return false
    return capabilities.hasCapability( capability = NetworkCapabilities.NET_

}
```

Рисунок 29 - fragment_feed.xml.

4.5 Реализация уведомлений

После успешной синхронизации данных приложение отправляет системное уведомление с текстом "Новые данные получены".

Для этого был создан канал уведомлений и вспомогательный класс NotificationHelper.

```
private const val NOTIFICATION_ID = 1001

1 Usage
fun createChannel(context: Context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            CHANNEL_ID,
            CHANNEL_NAME,
            importance = NotificationManager.IMPORTANCE_DEFAULT
        ).apply {
            description = CHANNEL_DESC
        }

        val manager =
            context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        manager.createNotificationChannel(channel)
    }
}

1 Usage
fun showSyncSuccess(context: Context) {
    try {
        val notification = NotificationCompat.Builder(context, channelId = CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_launcher_foreground)
            .setContentTitle("Синхронизация завершена")
            .setContentText("Новые данные получены")
            .setAutoCancel(true)
            .build()

        NotificationManagerCompat.from(context)
            .notify(NOTIFICATION_ID, notification)
    } catch (e: Exception) {
    }
}
```

Рисунок 30 - fragment_feed.xml.

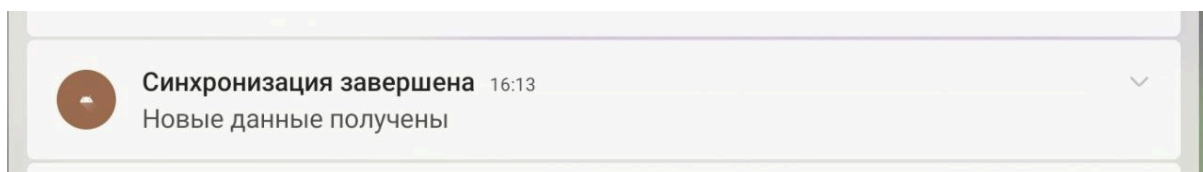


Рисунок 31 - Уведомление.

4.6 Запрос разрешений и обработка системных событий

При запуске приложения выполняется запрос разрешения на отправку уведомлений (POST_NOTIFICATIONS) для устройств с Android 13 и выше.

Также приложение корректно реагирует на отсутствие сети, откладывая синхронизацию до восстановления соединения.

```
class MainActivity : AppCompatActivity() {  
  
    3 Usages  
    private lateinit var binding: ActivityMainBinding  
    2 Usages  
    private lateinit var navController: NavController  
  
    1 Usage  
    private val requestPermissionLauncher =  
        registerForActivityResult( contract = ActivityResultContracts.RequestPermission() ) { granted ->  
        }  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        val navHostFragment =  
            supportFragmentManager.findFragmentById( id = R.id.nav_host_fragment ) as NavHostFragment  
  
        navController = navHostFragment.navController  
        binding.bottomNavigation.setupWithNavController(navController)  
  
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {  
            requestPermissionLauncher.launch( input = Manifest.permission.POST_NOTIFICATIONS )  
        }  
    }  
}
```

Рисунок 32 - MainActivity.kt.

4.7 Выводы

В рамках данной лабораторной работы был существенно улучшен пользовательский интерфейс приложения, реализована поддержка интерактивных элементов, фоновой синхронизации и уведомлений.

Приложение соответствует принципам MVVM, корректно работает в фоне и устойчиво к изменению состояния сети.

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсовой работы была осуществлена всесторонняя разработка мобильного приложения "Мессенджер" для операционной системы Android. Итогом проделанной работы стало создание функционального программного решения, удовлетворяющего поставленным требованиям и соответствующего современным подходам к архитектуре мобильных приложений. В ходе реализации были успешно решены задачи, связанные с построением навигации, управлением состоянием пользовательского интерфейса, а также с организацией устойчивого взаимодействия с сетевыми и локальными источниками данных.

Ключевым аспектом проекта стало практическое применение архитектурного паттерна MVVM, а также принципов реактивного программирования. Четкое разделение ответственности между слоями представления, бизнес-логики и доступа к данным позволило повысить надежность приложения и обеспечить корректную работу при изменениях конфигурации устройства. Использование библиотек Room и Retrofit в сочетании с корутинами способствовало эффективной обработке данных, высокой отзывчивости интерфейса и оптимальной производительности приложения.

Отдельное внимание в работе было уделено обеспечению автономности приложения. Реализация репозитория как единой точки доступа к данным и внедрение механизма фоновой синхронизации с использованием WorkManager позволили обеспечить доступность контента при нестабильном или отсутствующем интернет-соединении. Дополнение функциональности системой уведомлений и обработкой разрешений повысило интерактивность приложения и обеспечило его соответствие актуальным требованиям безопасности платформы Android. Архитектура проекта допускает дальнейшее масштабирование, включая

возможность реализации полноценного обмена сообщениями и системы авторизации пользователей.

В результате можно сделать вывод, что все цели и задачи, поставленные на этапе проектирования курсовой работы, были успешно достигнуты. Полученный опыт охватывает полный цикл разработки мобильного приложения - от проектирования пользовательского интерфейса до настройки фоновых процессов и работы с данными. Разработанное приложение "Мессенджер" представляет собой законченный прототип, демонстрирующий эффективное использование технологий Kotlin и Java и обладающий потенциалом для дальнейшего функционального расширения.